Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

Modelo de Relatório Técnico

ELE3515 - Grupo 03 - Máquina de Troco - Implementação

Líder: Mateus Macedo Nogueira
Redator: Ananda Dantas Fortunato
Debatedor: Daniel Medeiros Soares Carneiro

Natal, 25 de outubro de 2026

Copyright © 2025, Universidade Federal do Rio Grande do Norte.

Nenhuma parte deste material, sem autorização prévia por escrito, poderá ser reproduzida ou transmitida.

Como citar.
Dias, S. M. Material dos cursos de sistemas digitais (ELE3515): Modelo de Relatório Técnico, UFRN, Natal, RN, 2025, 17 p.

Resumo

Este relatório apresenta a etapa de implementação em VHDL de uma máquina de troco, a partir do projeto em nível de transferência entre registradores (RTL) desenvolvida na etapa anterior. A máquina recebe um valor de até R$ 10, em centavos, e devolve em moedas de R$ 1,00, R$ 0,50, R$ 0,25, R$ 0,10, R$ 0,05 e R$ 0,01, liberando uma moeda por vez e considerando os cofres vazios ou desabilitados. Os elementos de memória foram construídos a partir de flip-flops D e as decisões do circuito foram implementadas por multiplexadores descritos por suas equações booleanas. O sistema foi organizado em blocos — somador, subtrator, comparador de magnitude, multiplexadores, registradores, blocos combinacionais do *Dataway* e bloco de controle, cada um descrito, simulado e validado individualmente por meio de plots dos gráficos no Modelsim antes da integração e da programação no kit DE2. Os resultados das simulações confirmaram o comportamento previsto no projeto para cada bloco.

Palavras-chaves: Máquina de troco; Dataway; Projeto RTL; Multiplexadores, Registradores, Led.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Sumário

Resumo **2**

1 Introdução **5**

2 Fundamentação Teórica **6**

3 Correções do Projeto na Implementação **7**

4 Desenvolvimento da Implementação **8**
4.1 Multiplexadores (Mux) 8
4.1.1 Mux 2:1 8
4.1.2 Mux 8:1 9
4.2 Somador de 10 bits 10
4.3 Subtrator de 10 bits 11
4.4 Registradores 12
4.4.1 Registrador Vtot 12
4.4.2 Registrador do Banco Provisório 13
4.4.3 Registrador do Banco de Referência 14
4.4.4 Registrador do LED de Saída das Moedas 15
4.5 Comparador de 10 bits 16
4.6 Comparador de Magnitude 16
4.7 Comparador Vtot 17
4.8 Contador de Moedas 18
4.9 Gerador de Operação 19
4.10 Verificador de Operação 20
4.11 Trocador 21
4.12 Dataway 22
4.13 Máquina de Estados 23
4.14 Máquina de Troco FINAL 23

5 Conclusão **25**

Referências **26**

B Relato semanal **27**
B.1 Equipe 27
B.2 Defina o problema 27
B.3 Registro do brainstorming 27
B.4 Pontos-chaves 27
B.5 Questões de Pesquisa 28

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

B.6 Planejamento de pesquisa 28

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Capítulo 1

Introdução

Na etapa anterior, foi desenvolvido, por meio da metodologia RTL, o projeto de uma máquina de troco, compreendendo a máquina de estados de alto nível, o bloco operacional (*Dataway*) e a máquina de estados de baixo nível da unidade de controle. Esta etapa tem como objetivo implementar o circuito em VHDL, verificar seu funcionamento por simulação e prepará-lo para síntese e programação no kit de desenvolvimento DE2.

O sistema recebe um valor binário V, expresso em centavos, e determina sua composição utilizando moedas de R$ 1,00, R$ 0,50, R$ 0,25, R$ 0,10, R$ 0,05 e R$ 0,01. A operação é iniciada pelo acionamento do botão sincronizado T, e a seleção prioriza a moeda de maior valor disponível que não ultrapasse o saldo restante. Durante o processamento, novas solicitações são ignoradas e a saída L permanece piscando. Quando um cofre está vazio ou desabilitado ($c_x = 0$), sua denominação é desconsiderada nas próximas seleções.

A confirmação do troco ocorre somente quando o valor solicitado é integralmente composto. Para isso, as retiradas são registradas inicialmente no banco provisório, e o banco de referência é atualizado apenas no sucesso da operação. Caso o procedimento não consiga completar o valor, o estoque de referência é preservado, a saída *Return_ out* sinaliza a devolução e L permanece acesa até o carregamento de uma nova solicitação.

A implementação foi distribuída entre os integrantes do grupo por blocos funcionais, descritos em arquivos próprios e simulados individualmente antes da integração. Este relatório apresenta o funcionamento desses blocos, as correções necessárias em relação ao projeto original e a análise dos diagramas temporais utilizados para verificar o comportamento do sistema.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Capítulo 2

Fundamentação Teórica

A implementação de sistemas digitais em nível de transferência entre registradores, denominada RTL (*Register-Transfer Level*), fundamenta-se na organização de componentes capazes de armazenar, processar e transferir informações. Conforme a abordagem de Frank Vahid, essa organização compreende um caminho de dados e uma unidade de controle. O caminho de dados realiza as operações, enquanto a unidade de controle determina sua sequência por meio de sinais de comando, considerando as condições produzidas pelo próprio circuito. Essa separação permite converter o comportamento especificado no projeto em uma estrutura de hardware.

Os componentes dessa estrutura dividem-se em circuitos combinacionais e sequenciais. Nos combinacionais, as saídas dependem dos valores atuais das entradas, após o tempo de propagação. Multiplexadores selecionam dados, comparadores verificam relações entre valores e circuitos aritméticos executam operações como soma e subtração. Os circuitos sequenciais armazenam informações, permitindo que o comportamento também dependa de valores anteriores. Os registradores são formados por conjuntos de flip-flops e armazenam palavras binárias, sendo elementos fundamentais do caminho de dados.

Em um circuito síncrono, as atualizações dos registradores ocorrem na borda ativa do clock. A lógica conectada às suas entradas determina o próximo valor armazenado, possibilitando operações de carga, retenção e atualização. A coordenação dessas operações pode ser realizada por uma máquina de estados finitos, composta por memória de estado e lógica combinacional. O estado atual e as condições de entrada determinam as transições, enquanto os sinais de controle comandam os componentes do caminho de dados.

A linguagem VHDL permite descrever a interface e o funcionamento dos componentes digitais, além de suas conexões. A organização hierárquica favorece a divisão do sistema em blocos menores, que podem ser implementados e verificados individualmente antes da integração. Durante a etapa final, o Quartus converte a descrição em VHDL em uma estrutura de elementos lógicos e registradores. Em seguida, as etapas de roteamento definem quais recursos da FPGA serão utilizados e como serão interligados. A atribuição dos pinos, realizada no *Pin Planner*, associa as entradas e saídas do circuito aos componentes da placa, como chaves, botões, LEDs e fonte de clock.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Capítulo 3

Correções do Projeto na Implementação

Durante a implementação em VHDL, foram identificadas inconsistências no projeto original. As correções realizadas são apresentadas a seguir:

- **Comparador Vtot:** substituição da comparação estrita $V_{tot} > V_m$ pela condição $V_{tot} \geq V_m$.
- **Verificador de Operação:** substituição de empty por sua negação nas seis expressões de Operate.
- **Registrador Vtot:** reorganização das entradas do multiplexador e correção das equações de seleção para executar limpeza, retenção, subtração e carga, com prioridade da carga sobre a subtração.
- **Constantes do Trocador:** adequação dos valores nas entradas do multiplexador à correspondência definida pela tabela de seleção das moedas.
- **Seletores do Trocador:** compatibilização das convenções de seleção mediante o cruzamento das conexões S0 e S2 entre o codificador e o multiplexador.
- **Transição para erro:** retirada da exigência de banco vazio, permitindo reconhecer erro sempre que houver saldo restante sem moeda utilizável.
- **LED geral L:** implementação do piscar durante o processamento e inclusão de memória para manter a indicação de erro até a reinicialização ou o carregamento de um novo valor V.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Capítulo 4

Desenvolvimento da Implementação

4.1 Multiplexadores (Mux)

4.1.1 Mux 2:1

O multiplexador 2×1 de 10 bits tem a função de selecionar uma entre duas entradas binárias e encaminhá-la à saída. O bloco possui duas entradas de dados, I0 e I1, uma entrada de seleção, Sel, e uma saída de 10 bits, D. Quando Sel está em nível lógico baixo, a saída recebe I0; quando está em nível alto, recebe I1.

A implementação utiliza operações lógicas AND, OR e NOT aplicadas individualmente aos dez bits. Para cada posição, I0 é combinada com a negação de Sel, enquanto I1 é combinada com Sel. A operação OR reúne os dois termos, fazendo com que apenas a entrada selecionada determine a saída. As atribuições são concorrentes e descrevem um circuito combinacional, sem armazenamento ou dependência de clock.

A Figura 4.1 apresenta a simulação do bloco, com os valores dos barramentos exibidos em decimal. Inicialmente, I0 recebe 100, I1 recebe 999 e Sel permanece em nível baixo, resultando em D igual a 100. Em 10 ns, Sel passa para nível alto e a saída assume o valor 999. Em 20 ns, mantendo Sel em nível alto, I1 é alterada para 25, e a saída acompanha essa mudança. Finalmente, em 30 ns, Sel retorna ao nível baixo e I0 é alterada para zero, fazendo com que D também assuma zero.

Figura 4.1: Simulação do multiplexador 2×1 de 10 bits.



Fonte: Daniel Medeiros.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Os resultados confirmam a seleção das duas entradas e o acompanhamento das alterações da entrada selecionada. Assim, o multiplexador apresentou o comportamento esperado nas condições simuladas.

4.1.2 Mux 8:1

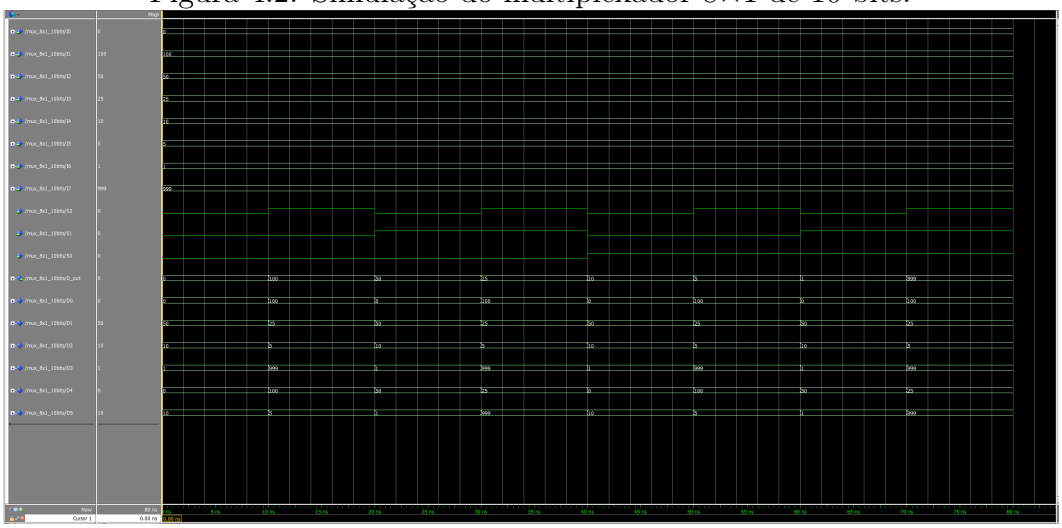
O multiplexador 8×1 de 10 bits tem a função de selecionar uma entre oito entradas binárias e encaminhá-la à saída. O bloco possui oito entradas de dados, de I0 a I7, três sinais de seleção, S2, S1 e S0, e uma saída de 10 bits, denominada D_out. Trata-se de um circuito combinacional, cuja saída depende das entradas e da combinação aplicada aos seletores.

A implementação utiliza sete multiplexadores 2×1 de 10 bits, organizados em três níveis. No primeiro nível, quatro multiplexadores selecionam entre os pares I0/I1, I2/I3, I4/I5 e I6/I7, sob controle de S2, produzindo os sinais D0 a D3. No segundo nível, dois multiplexadores controlados por S1 selecionam esses resultados e produzem D4 e D5. No terceiro nível, um multiplexador controlado por S0 determina a saída final.

Nessa organização, S2 funciona como o bit menos significativo da seleção, enquanto S0 funciona como o mais significativo. Portanto, a sequência de seleção das entradas I0 a I7 corresponde às combinações de 000 a 111 consideradas na ordem S0, S1 e S2. Essa correspondência deve ser respeitada nas conexões com os demais blocos.

A Figura 4.8 apresenta a simulação, com as entradas I0 a I7 fixadas, respectivamente, em 0, 100, 50, 25, 10, 5, 1 e 999. As combinações dos seletores são alteradas a cada 10 ns, percorrendo todas as entradas. Assim, a saída apresenta sucessivamente os valores 0, 100, 50, 25, 10, 5, 1 e 999, nos intervalos iniciados em 0, 10, 20, 30, 40, 50, 60 e 70 ns. Os sinais intermediários D0 a D5 também permitem acompanhar a seleção realizada em cada nível.

Figura 4.2: Simulação do multiplexador 8×1 de 10 bits.



Fonte: Daniel Medeiros.

Os resultados confirmam o encaminhamento das oito entradas à saída conforme as combinações aplicadas. Dessa forma, o bloco apresentou o comportamento esperado para todas as combinações de seleção testadas, respeitando a ordem dos seletores definida na implementação.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

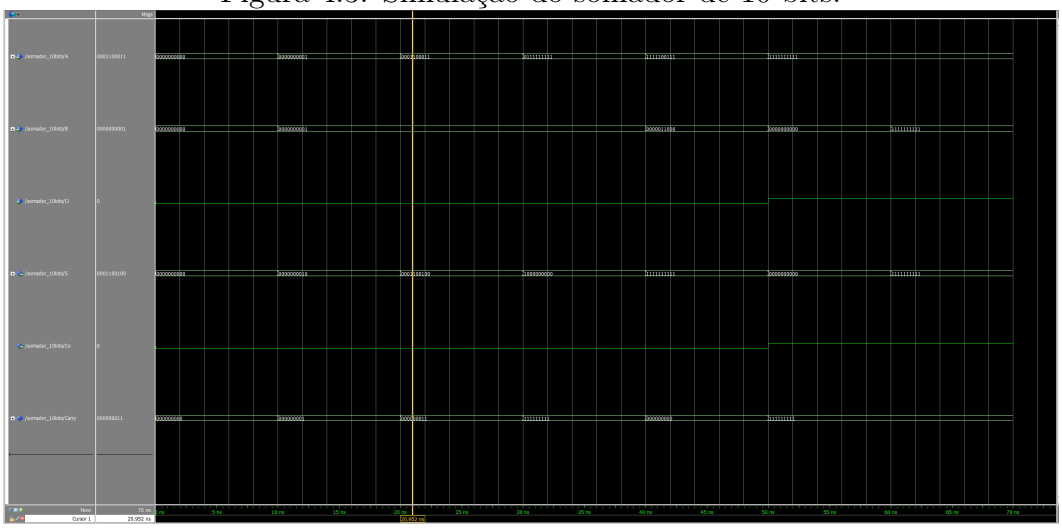
4.2 Somador de 10 bits

O somador de 10 bits realiza a adição de duas entradas binárias, A e B, considerando também o transporte de entrada Ci. O resultado é disponibilizado na saída S, de 10 bits, enquanto Co representa o transporte de saída. Para valores interpretados como inteiros sem sinal, a operação completa satisfaz a relação $A + B + Ci = S + 1024\,Co$, permitindo representar resultados que ultrapassam o limite de 1023 da saída S.

A implementação utiliza dez somadores completos de 1 bit conectados em cascata, formando um somador com propagação de transporte. O primeiro estágio soma os bits menos significativos de A e B com Ci. Os estágios seguintes recebem o transporte produzido pelo estágio anterior, por meio do vetor interno Carry. O último estágio calcula o bit mais significativo de S e fornece Co. A estrutura é combinacional, sem armazenamento ou dependência de clock, e seu tempo de resposta está associado à propagação dos sinais pelos estágios.

A Figura 4.3 apresenta a simulação, com os barramentos exibidos em binário. Inicialmente, as entradas são nulas e o resultado é zero. Em 10 ns, a operação $1 + 1$ produz 2. Em 20 ns, a soma $99 + 1$ resulta em 100. Em 30 ns, a operação $511 + 1$ produz 512, exercitando a propagação do transporte pelos estágios inferiores. Em 40 ns, a soma $999 + 24$ resulta em 1023, mantendo Co em zero. Em 50 ns, são aplicados $A = 1023$, $B = 0$ e $Ci = 1$, produzindo S igual a zero e Co igual a um, correspondentes ao resultado completo 1024. Finalmente, em 60 ns, a operação $1023 + 1023 + 1$ resulta em 2047, representado por S igual a 1023 e Co igual a um.

Figura 4.3: Simulação do somador de 10 bits.



Fonte: Daniel Medeiros.

Os resultados confirmam a realização das adições, a participação do transporte de entrada e a indicação do transporte de saída quando o resultado ultrapassa dez bits. Dessa forma, o circuito apresentou o comportamento esperado nas condições simuladas.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

4.3 Subtrator de 10 bits

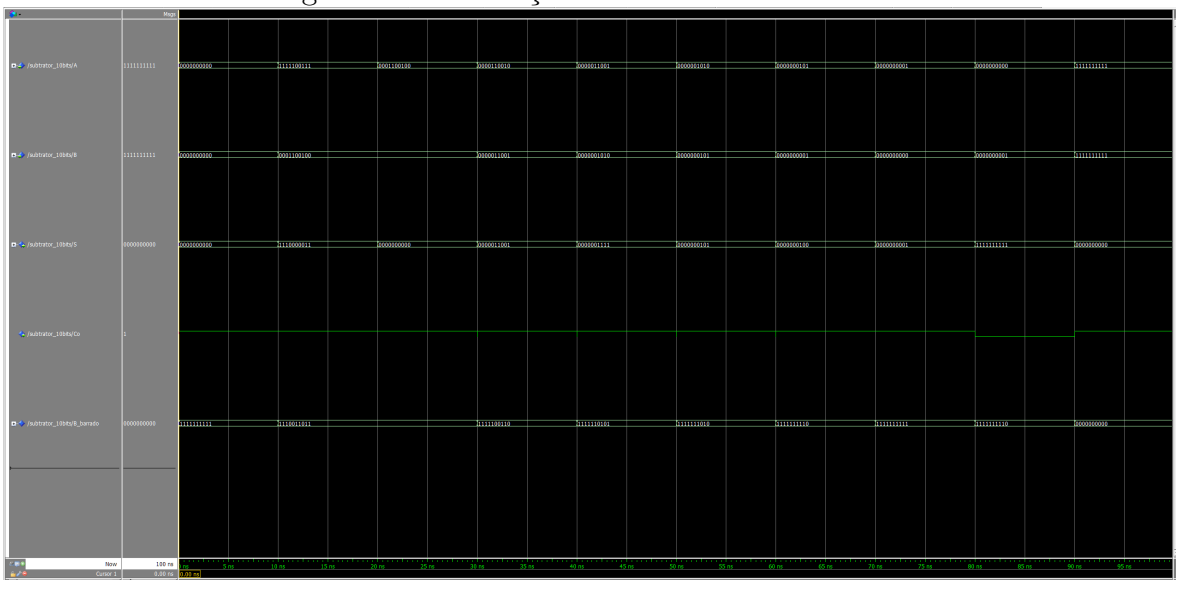
O subtrator de 10 bits realiza a operação $A - B$, recebendo duas palavras binárias e disponibilizando o resultado na saída S. No caminho de dados da máquina de troco, essa operação permite calcular o saldo restante após a seleção de uma moeda. O bloco também fornece Co, correspondente ao transporte de saída do somador utilizado na implementação.

A implementação emprega a subtração por complemento de dois, utilizando a identidade $A - B = A + \overline{B} + 1$. Inicialmente, os dez bits de B são invertidos, produzindo o sinal interno B_barrado. Em seguida, um somador de 10 bits recebe A, B_barrado e o transporte de entrada fixado em um. O circuito é combinacional e reutiliza o somador apresentado anteriormente. Para operandos interpretados como valores sem sinal, Co igual a um indica ausência de empréstimo, enquanto Co igual a zero indica que $A < B$.

A Figura 4.4 apresenta a simulação, com os barramentos exibidos em binário. Inicialmente, a operação $0 - 0$ produz zero. Em 10 ns, a subtração $999 - 100$ resulta em 899. Em 20 ns, $100 - 100$ produz zero, confirmando a operação com entradas iguais. Nos instantes de 30, 40, 50 e 60 ns, são realizadas, respectivamente, as operações $50 - 25$, $25 - 10$, $10 - 5$ e $5 - 1$, produzindo os resultados 25, 15, 5 e 4. Em 70 ns, a operação $1 - 0$ mantém o valor um.

Em 80 ns, aplica-se $0 - 1$, produzindo S igual a 1111111111 e Co igual a zero. Esse padrão corresponde a -1 em complemento de dois ou a 1023 quando interpretado sem sinal, pois a saída de dez bits representa o resultado módulo 1024. Na máquina de troco, a seleção das moedas deve impedir essa situação, autorizando apenas subtrações em que o valor da moeda não ultrapasse o saldo. Finalmente, em 90 ns, a operação $1023 - 1023$ resulta em zero, com Co igual a um.

Figura 4.4: Simulação do subtrator de 10 bits.



Fonte: Daniel Medeiros.

Os resultados confirmam o cálculo das diferenças e a indicação de ausência ou ocorrência de empréstimo.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

4.4 Registradores

Os três primeiros registradores possuem a mesma estrutura: cada bit é guardado por uma célula composta por um multiplexador 4×1, que escolhe o próximo valor do bit, e pelo flip-flop D fornecido pela disciplina, que armazena esse valor na borda de subida do clock. Essa célula foi descrita uma única vez (`fatia_reg`) e instanciada uma vez para cada bit. Os sinais de seleção s_1 e s_0 são comuns a todas as células de um registrador, de modo que todos os bits realizam a mesma operação no mesmo ciclo. As entradas assíncronas *set* e *clr* do flip-flop são mantidas em nível 1, pois a inicialização é feita pelos multiplexadores.

4.4.1 Registrador Vtot

O registrador Vtot guarda, em 10 bits, o valor que ainda falta ser trocado. Ele precisa realizar quatro operações: zerar (inicialização), carregar o valor V das chaves (estado V), receber o resultado $V_{tot} - V_M$ calculado pelo Trocador (estado S) e manter o valor atual. Na implementação, as entradas do multiplexador de cada fatia foram ligadas da seguinte forma: $i_0 = 0$, $i_1 = Q$, $i_2 = V_{tot} - V_M$ e $i_3 = V$.

O sinal $CLEAR$ é ativo em nível baixo, como descrito na estrutura geral do *Dataway*, e a carga de V tem prioridade sobre a atualização pela subtração.A Tabela 4.1 resume o comportamento.

Tabela 4.1: Operações do registrador Vtot.

$CLEAR$	$LOAD$	DEC	s_1	s_0	Operação
0	X	X	0	0	$V_{tot} \leftarrow 0$
1	0	0	0	1	$V_{tot} \leftarrow V_{tot}$ (mantém)
1	0	1	1	0	$V_{tot} \leftarrow V_{tot} - V_M$
1	1	X	1	1	$V_{tot} \leftarrow V$

A partir da tabela, os sinais de seleção são:

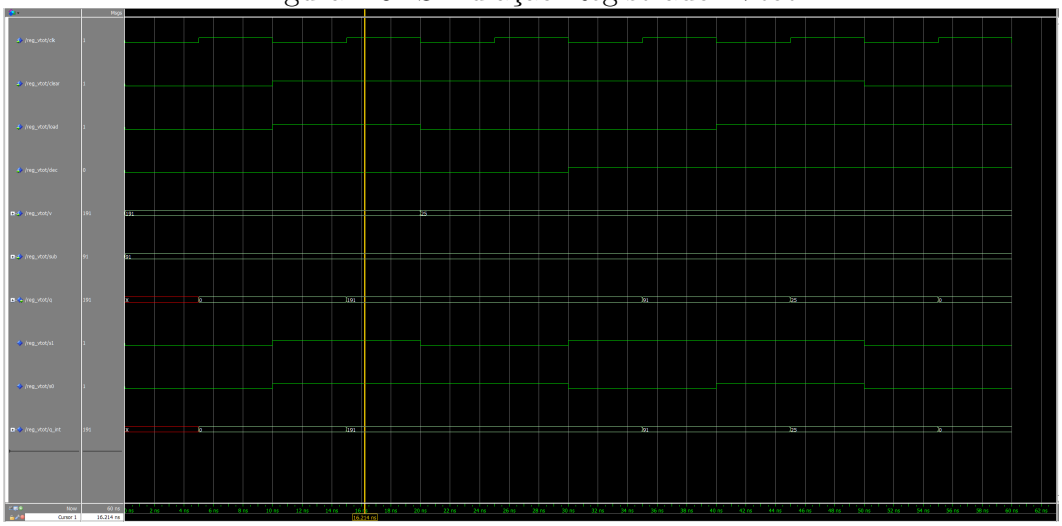
$$s_1 = CLEAR \cdot (LOAD + DEC), \tag{4.1}$$

$$s_0 = CLEAR \cdot (LOAD + \overline{DEC}). \tag{4.2}$$

No *Dataway*, a entrada $LOAD$ é ligada ao sinal $LOAD_V$ da máquina de estados, e a entrada DEC recebe o sinal $CHANGE$ barrado, de modo que a subtração é registrada no estado S, como previsto na MDE de alto nível.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Figura 4.5: Simulação Registrador Vtot.



Fonte: Daniel Medeiros.

4.4.2 Registrador do Banco Provisório

O registrador do banco provisório armazena, em 4 bits, a quantidade de moedas de um tipo durante o cálculo do troco. Há uma instância para cada moeda. Seguindo a Seção 4.2.6 do relatório de projeto, as entradas de cada fatia são: $i_0 = Q$ (mantém), $i_1 = V_{REF}$ (carrega o banco de referência), $i_2 = Q - 1$ (decrementa) e $i_3 = 0$ (não utilizada). O decremento só ocorre quando DEC e $OPERATE$ estão ativos, isto é, quando é o momento de atualizar os bancos e a moeda em questão foi realmente usada, segundo o Verificador de Operações.

O valor $Q - 1$ é gerado dentro do próprio registrador por um decrementador combinacional de 4 bits. Subtrair 1 equivale a inverter o bit menos significativo e propagar um "empréstimo" (*borrow*) para os bits seguintes enquanto os bits anteriores forem 0:

$$D_0 = \overline{Q_0}, \qquad b_0 = \overline{Q_0}, \tag{4.3}$$

$$D_1 = Q_1 \oplus b_0, \qquad b_1 = b_0 \cdot \overline{Q_1}, \tag{4.4}$$

$$D_2 = Q_2 \oplus b_1, \qquad b_2 = b_1 \cdot \overline{Q_2}, \tag{4.5}$$

$$D_3 = Q_3 \oplus b_2. \tag{4.6}$$

Tabela 4.2: Operações do registrador do banco provisório.

$CLEAR$	$LOAD$	DEC	$OPERATE$	s_1	s_0	Operação
0	X	X	X	0	0	mantém
1	1	X	X	0	1	$B_{prov} \leftarrow B_{ref}$
1	0	1	1	1	0	$B_{prov} \leftarrow B_{prov} - 1$
1	0		outros	0	0	mantém

As equações de seleção são:

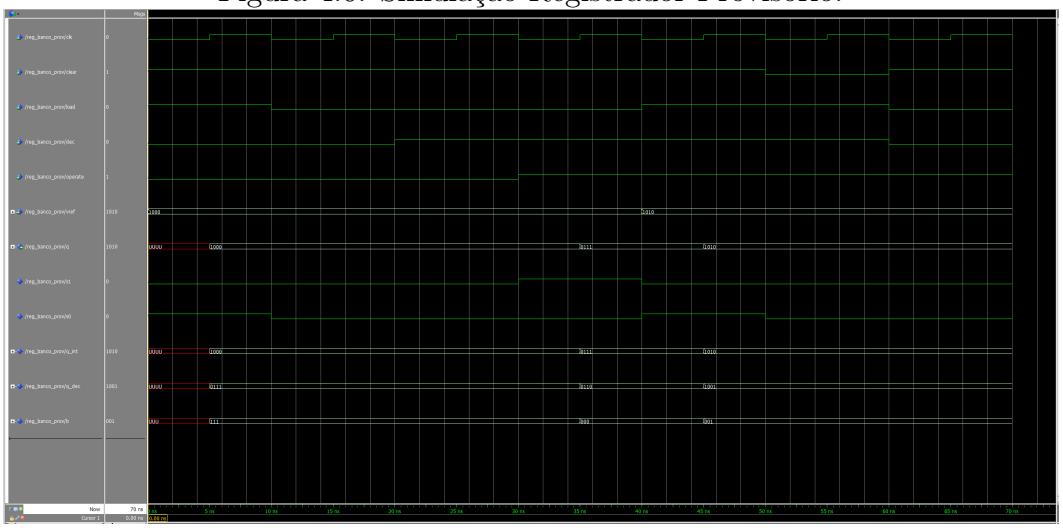
$$s_0 = CLEAR \cdot LOAD, \tag{4.7}$$

$$s_1 = CLEAR \cdot \overline{LOAD} \cdot DEC \cdot OPERATE. \tag{4.8}$$

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

A combinação $s_1 s_0 = 11$ nunca ocorre, pois exigiria $LOAD = 1$ e $LOAD = 0$ ao mesmo tempo.

Figura 4.6: Simulação Registrador Provisório.



Fonte: Daniel Medeiros.

4.4.3 Registrador do Banco de Referência

O registrador do banco de referência guarda a quantidade "oficial" de cada moeda, que só é alterada quando um troco é concluído com sucesso (estado L). As entradas de cada fatia são $i_0 = 1$, $i_1 = Q$, $i_2 = PROV$ e $i_3 = 0$. Com $CLEAR$ em nível baixo (estado R), todos os bits vão a 1 e o banco assume o valor $1111_2 = 15$, que corresponde ao reabastecimento com 15 moedas.

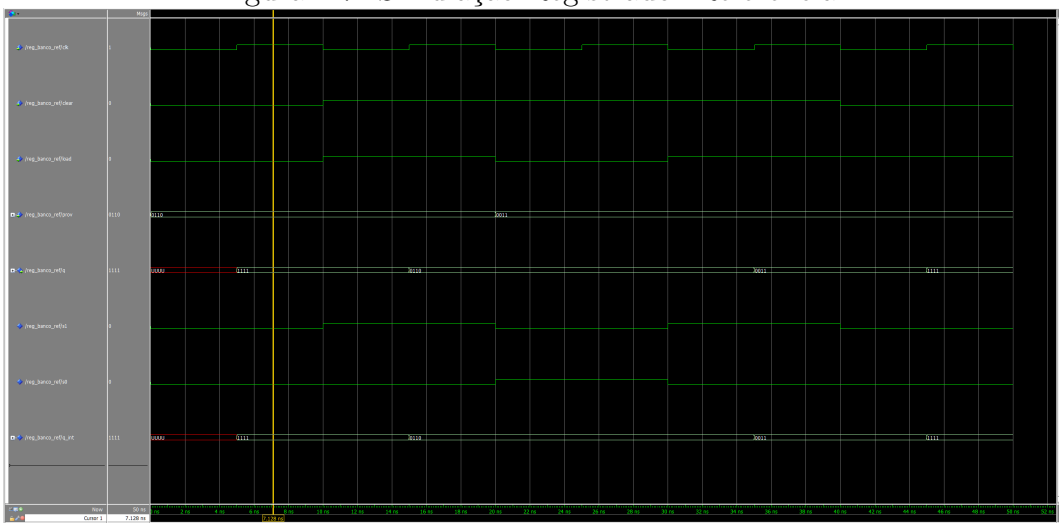
Tabela 4.3: Operações do registrador do banco de referência.

$CLEAR$	$LOAD$	s_1	s_0	Operação
0	X	0	0	$B_{ref} \leftarrow 1111$ (reabastece)
1	0	0	1	$B_{ref} \leftarrow B_{ref}$ (mantém)
1	1	1	0	$B_{ref} \leftarrow B_{prov}$

Os sinais de seleção são $s_1 = CLEAR \cdot LOAD$ e $s_0 = CLEAR \cdot \overline{LOAD}$.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Figura 4.7: Simulação Registrador Referência.



Fonte: Daniel Medeiros.

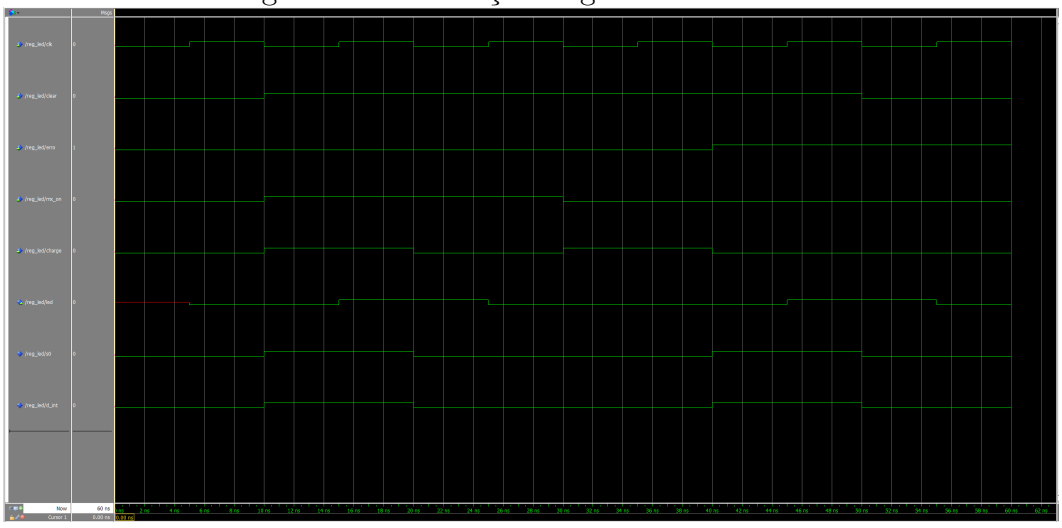
4.4.4 Registrador do LED de Saída das Moedas

O registrador do LED possui 1 bit e há uma instância para cada moeda (saídas i_1 a i_6). Ele é formado por um multiplexador 2x1 e um flip-flop D. A entrada 0 do multiplexador recebe o nível lógico 0 (LED apagado) e a entrada 1 recebe o nível lógico 1 (LED aceso). O sinal de seleção é:

$$S_0 = CLEAR \cdot (ERRO + MX_ON \cdot CHARGE)\,. \tag{4.9}$$

Durante o troco, $CHARGE$ (sinal $CHANGE$ da máquina de estados) vale 1 no estado T e 0 no estado S. Assim, se a moeda x é a escolhida ($MX_ON = 1$), o LED acende no estado T e apaga no estado S, produzindo um pulso por moeda liberada. Quando $ERRO = 1$, o LED permanece aceso enquanto o erro estiver ativo.

Figura 4.8: Simulação Registrador do LED.



Fonte: Daniel Medeiros.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

4.5 Comparador de 10 bits

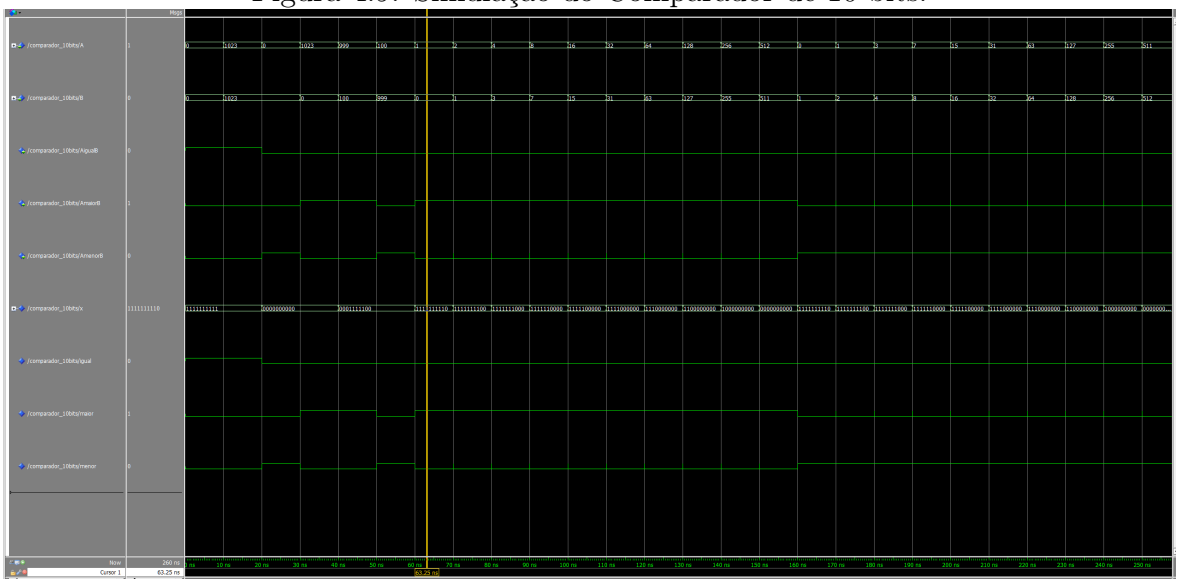
O comparador de 10 bits verifica a relação entre duas entradas binárias sem sinal, A e B. O bloco possui três saídas: AigualB, ativada quando os valores são iguais; AmaiorB, ativada quando A é maior que B; e AmenorB, ativada quando A é menor que B. Para entradas binárias definidas, apenas uma dessas saídas permanece em nível lógico alto.

A implementação utiliza operações lógicas, constituindo um circuito combinacional. Inicialmente, o vetor interno x identifica a igualdade entre os bits correspondentes de A e B, por meio da negação da operação XOR. A igualdade entre as entradas é determinada pela operação AND entre todos os bits de x. As condições de maior e menor são obtidas pela análise das posições, da mais significativa para a menos significativa. Uma posição somente determina o resultado quando todos os bits anteriores, de maior peso, são iguais.

A Figura 4.9 apresenta a simulação, com as entradas A e B exibidas em decimal. Nos intervalos de 0 a 10 ns e de 10 a 20 ns, são aplicados, respectivamente, os pares $(0, 0)$ e $(1023, 1023)$, mantendo AigualB em nível alto. Em 20 ns, a comparação entre zero e 1023 ativa AmenorB; em 30 ns, a inversão desses valores ativa AmaiorB. Em 40 ns, o par $(999, 100)$ mantém a indicação de A maior que B, enquanto, em 50 ns, o par $(100, 999)$ ativa a indicação de A menor que B.

Entre 60 e 160 ns, são aplicados pares em que A corresponde a uma potência de dois e B ao valor imediatamente anterior, desde $(1, 0)$ até $(512, 511)$. Em todos esses casos, AmaiorB permanece ativa. Entre 160 e 260 ns, os operandos são invertidos, desde $(0, 1)$ até $(511, 512)$, mantendo AmenorB ativa. Esses testes mostram a decisão da comparação nas diferentes posições dos dez bits.

Figura 4.9: Simulação do Comparador de 10 bits.



Fonte: Daniel Medeiros.

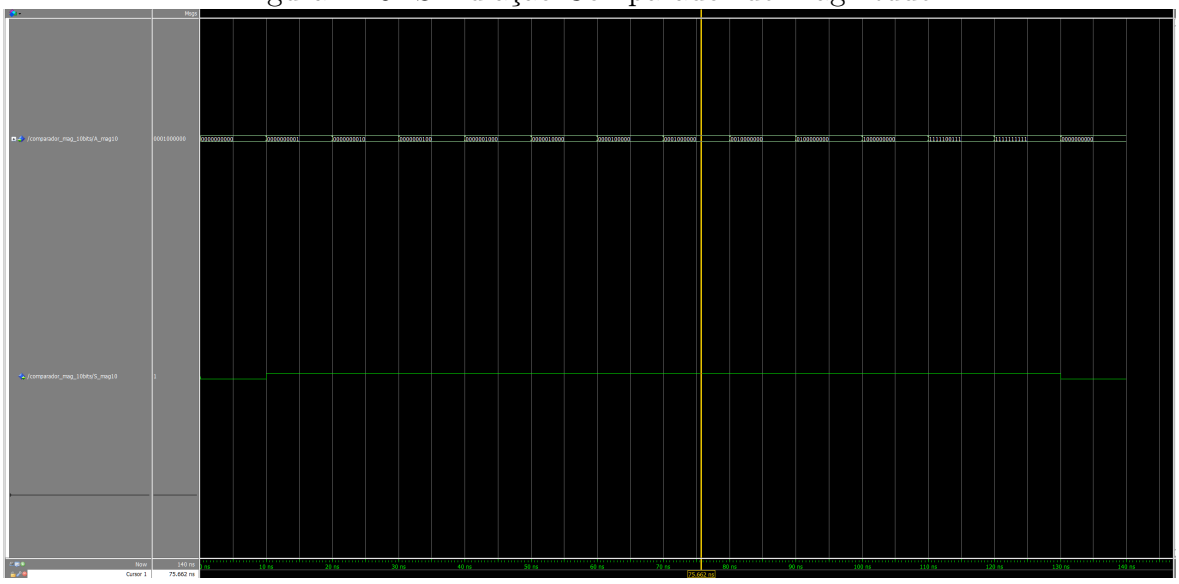
4.6 Comparador de Magnitude

A Figura 4.10 apresenta a simulação funcional do módulo Comparador_mag_10bits, um circuito puramente combinacional que determina se o valor de entrada A_mag10

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

(10 bits) é maior que zero. A saída é dada pela operação lógica OU entre todos os bits da entrada, $S = A_0 + A_1 + \cdots + A_9$, e portanto assume nível alto sempre que ao menos um bit de A_mag10 estiver em nível alto, e nível baixo somente quando $A = 0$. A simulação tem duração de 140 ns, com os estímulos variando em passos de 10 ns. Em 0 ns, a entrada vale 0000000000 e a saída permanece em nível baixo. A partir de 10 ns, são aplicados valores com um único bit em nível alto, deslocado uma posição a cada passo (0000000001, 0000000010, 0000000100, e assim por diante, até 1000000000 em 100 ns), o que verifica a sensibilidade da saída a cada um dos dez bits individualmente. Em todos esses casos, S_mag10 vai a nível alto e permanece assim. Em seguida, são aplicados os valores 1111100111 (999) em 110 ns e 1111111111 (1023) em 120 ns, com mais de um bit ativo, e a saída continua em nível alto. Por fim, em 130 ns, a entrada retorna a 0000000000 e a saída volta a nível baixo, confirmando que a saída depende apenas do valor atual da entrada. No instante do cursor (75 ns), a entrada vale 0001000000 (64) e S_mag10 está em nível alto, como previsto pela equação.

Figura 4.10: Simulação Comparador de Magnitude.



Fonte: Daniel Medeiros.

4.7 Comparador Vtot

A Figura 4.11 apresenta o resultado da simulação funcional do módulo Comparador_Vtot, composto por sete instâncias do componente Comparador_10bits que comparam, em paralelo, a entrada Vtot (10 bits) com as constantes 100, 50, 25, 10, 5, 1 e 0, associadas aos sinais Vtot_gt_Vm1 a Vtot_gt_Vm6 e VTOT_EQ_0. O estímulo foi aplicado em passos de 10 ns, totalizando 210 ns, e para cada limiar foram testados um valor imediatamente abaixo, um igual e um imediatamente acima (por exemplo, 99, 100 e 101 para $V_{m1} = 100$), seguidos dos valores extremos 999, 1000 e 1023. Em $V_{tot} = 0$, apenas VTOT_EQ_0 permanece em nível alto, retornando a zero a partir de 10 ns. Conforme V_{tot} cresce, as saídas Vtot_gt_Vm6 a Vtot_gt_Vm1 são ativadas em sequência e permanecem em nível alto até o fim da simulação, inclusive para 999, 1000 e 1023, sem indício de saturação ou *overflow*. Os sinais auxiliares eq_n apresentam um pulso de 10 ns exatamente quando $V_{tot} = V_{m_n}$, e os sinais gt_n são ativados apenas no valor seguinte. Observa-se, porém, que Vtot_gt_Vm_n é ativado já

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

na condição de igualdade (por exemplo, Vtot_gt_Vm5 sobe em $V_{tot} = 5$, instante de 40 ns), comportando-se como uma comparação "maior ou igual" e não "estritamente maior", o que coincide com o início do pulso de eq_n e precede a subida de gt_n.

Figura 4.11: Simulação Comparador Vtot.



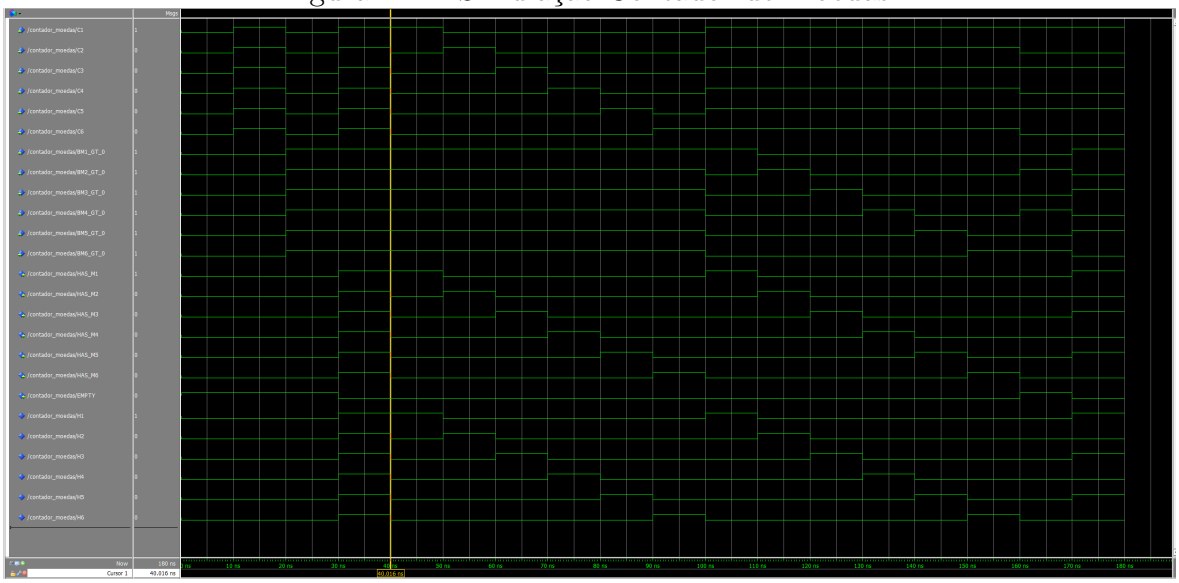
Fonte: Daniel Medeiros.

4.8 Contador de Moedas

A Figura 4.12 apresenta a simulação funcional do módulo Contador_moedas, um circuito puramente combinacional que determina, para cada uma das seis moedas, se há disponibilidade para uso. As entradas são os sinais de seleção C1 a C6 e os indicadores BM1_GT_0 a BM6_GT_0, estes provenientes dos comparadores de quantidade. Internamente, cada sinal H_n é obtido pela operação lógica E entre BM_n_GT_0 e C_n, ou seja, a moeda n é considerada disponível apenas quando há quantidade maior que zero e ela está selecionada. As saídas HAS_M1 a HAS_M6 replicam diretamente H_1 a H_6, e a saída EMPTY é definida como $\overline{H_1 + H_2 + H_3 + H_4 + H_5 + H_6}$, sendo ativada somente quando nenhuma moeda está disponível. A simulação tem duração de 180 ns, com os estímulos variando em passos de 10 ns. No instante do cursor (40 ns), C1 e todos os indicadores BM_n_GT_0 estão em nível alto, enquanto C2 a C6 estão em nível baixo; consequentemente, apenas H_1 e HAS_M1 estão ativos e EMPTY permanece em nível baixo, como previsto pelas equações. Ao longo da simulação, os sinais HAS_M_n e H_n apresentam formas de onda idênticas, e a ativação transita entre as moedas conforme a mudança das entradas C_n, com HAS_M1 desativando e HAS_M2 ativando em 50 ns. Em todos os instantes observados, EMPTY está em nível alto exatamente quando todos os sinais HAS_M_n estão em nível baixo, o que confirma o funcionamento da lógica de detecção de estado vazio.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Figura 4.12: Simulação Contador de Moedas.



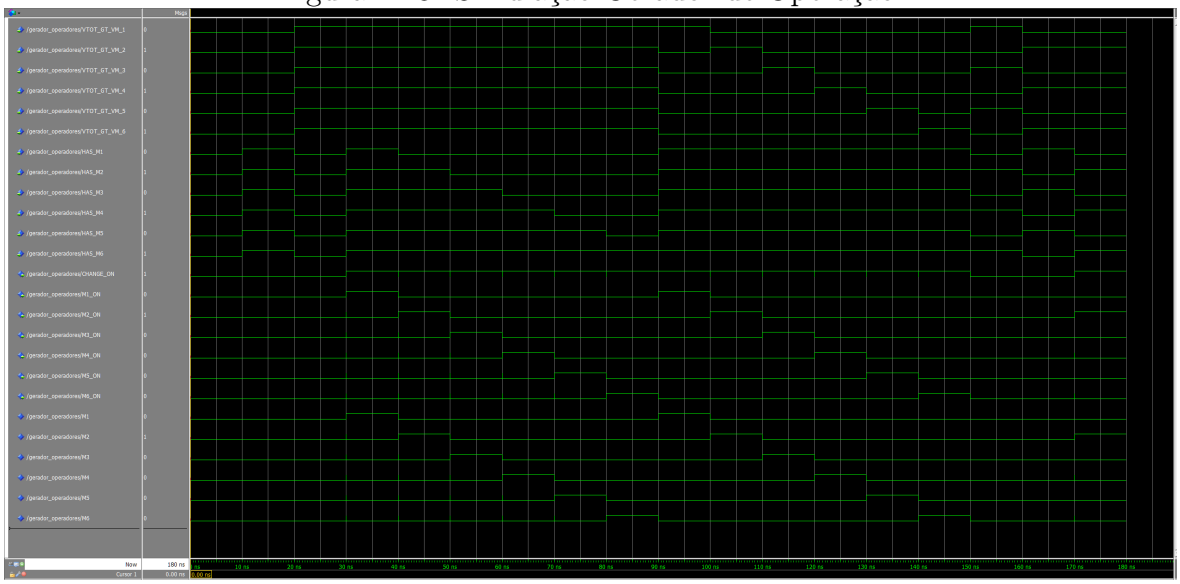
Fonte: Daniel Medeiros.

4.9 Gerador de Operação

A Figura 4.13 apresenta a simulação funcional do módulo gerador_operadores, um circuito puramente combinacional que seleciona qual moeda deve ser utilizada com base em um critério de prioridade. As entradas são os sinais VTOT_GT_VM_1 a VTOT_GT_VM_6, que indicam se o valor total V_{tot} é maior que o limiar de cada moeda, e os sinais HAS_M1 a HAS_M6, que indicam a disponibilidade de cada moeda. Internamente, a moeda n é escolhida quando VTOT_GT_VM_n e HAS_M$_n$ estão em nível alto e nenhuma moeda de índice menor foi escolhida, isto é, $M_n = VTOT_GT_VM_n \cdot HAS_M_n \cdot \overline{M_1} \cdots \overline{M_{n-1}}$. Assim, M_1 possui a maior prioridade e M_6 a menor, de modo que no máximo uma das saídas M1_ON a M6_ON pode estar ativa por vez. A saída CHANGE_ON é dada por $M_1 + M_2 + \cdots + M_6$, sinalizando que alguma moeda foi selecionada. A simulação tem duração de 180 ns, com os estímulos variando em passos de 10 ns. No instante inicial (0 ns), as entradas VTOT_GT_VM_2, VTOT_GT_VM_4 e VTOT_GT_VM_6, bem como HAS_M2, HAS_M4 e HAS_M6, estão em nível alto, tornando as moedas 2, 4 e 6 candidatas simultâneas. Apesar disso, apenas M2_ON é ativada, pois M_2 tem prioridade sobre M_4 e M_6, e CHANGE_ON permanece em nível alto, o que comprova o funcionamento da lógica de prioridade. Ao longo da simulação, observa-se que as saídas M1_ON a M6_ON são ativadas uma de cada vez, em pulsos de 10 ns, sem nenhuma sobreposição, e que CHANGE_ON só vai a nível baixo no intervalo em que nenhuma delas está ativa. Os sinais internos M_1 a M_6 apresentam formas de onda idênticas às das saídas correspondentes, já que M$_n$_ON apenas replica M_n.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Figura 4.13: Simulação Gerador de Operação.



Fonte: Daniel Medeiros.

4.10 Verificador de Operação

O verificador de operação tem a função de autorizar a atualização dos bancos provisórios das moedas selecionadas, considerando o saldo restante e a disponibilidade do banco. O bloco recebe Vtot_eq_0, que indica saldo igual a zero, empty, que indica banco vazio, e o vetor M_on, que identifica a moeda selecionada. Sua saída é o vetor Operate, com uma autorização para cada uma das seis operações.

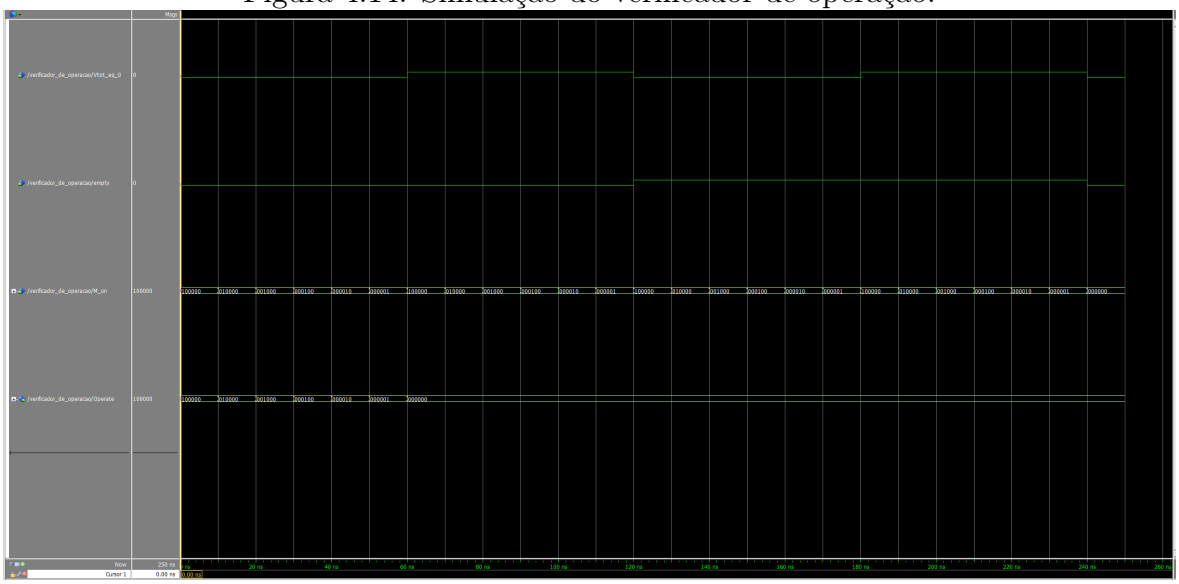
A implementação utiliza seis equações lógicas concorrentes. Cada bit de Operate resulta da operação AND entre a negação de Vtot_eq_0, a negação de empty e o bit correspondente de M_on. Assim, a autorização somente ocorre quando existe saldo restante, o banco não está vazio e a moeda está selecionada. Em relação à expressão original do projeto, a utilização de empty foi corrigida para sua negação, pois a operação deve ser permitida com o banco disponível, e não quando ele está vazio.

A Figura 4.14 apresenta a simulação do bloco. Entre 0 e 60 ns, Vtot_eq_0 e empty permanecem em zero, enquanto os seis bits de M_on são ativados individualmente, a cada 10 ns. Nesse intervalo, Operate acompanha os padrões 100000, 010000, 001000, 000100, 000010 e 000001, confirmando a autorização de cada posição.

Entre 60 e 120 ns, Vtot_eq_0 passa para um, bloqueando todas as saídas, mesmo com uma moeda selecionada. Entre 120 e 180 ns, o saldo volta a ser diferente de zero, mas empty permanece em um, mantendo Operate em 000000. Entre 180 e 240 ns, as duas condições de bloqueio estão ativas e as autorizações continuam desabilitadas. Finalmente, em 240 ns, Vtot_eq_0 e empty retornam a zero, mas M_on recebe 000000, confirmando que nenhuma operação é autorizada sem uma moeda selecionada.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Figura 4.14: Simulação do verificador de operação.



Fonte: Daniel Medeiros.

4.11 Trocador

O Trocador calcula o saldo restante após a utilização de uma moeda. O bloco recebe Vtot, que representa o valor atual do troco, e o vetor M_on, que identifica a denominação selecionada. A saída Vtot_menos_Vm fornece a diferença entre o saldo e o valor dessa moeda. O circuito apenas calcula o resultado; seu armazenamento é realizado pelo registrador Vtot.

A implementação reúne uma lógica de codificação, um multiplexador 8×1 de 10 bits e um subtrator de 10 bits. Os sinais de M_on são convertidos nos seletores S2, S1 e S0, responsáveis pela escolha do valor monetário. As entradas I1 a I6 do multiplexador recebem, respectivamente, 100, 50, 25, 10, 5 e 1 centavo, enquanto I0 e I7 recebem zero. O valor selecionado, denominado Vm_x, é encaminhado ao subtrator juntamente com Vtot.

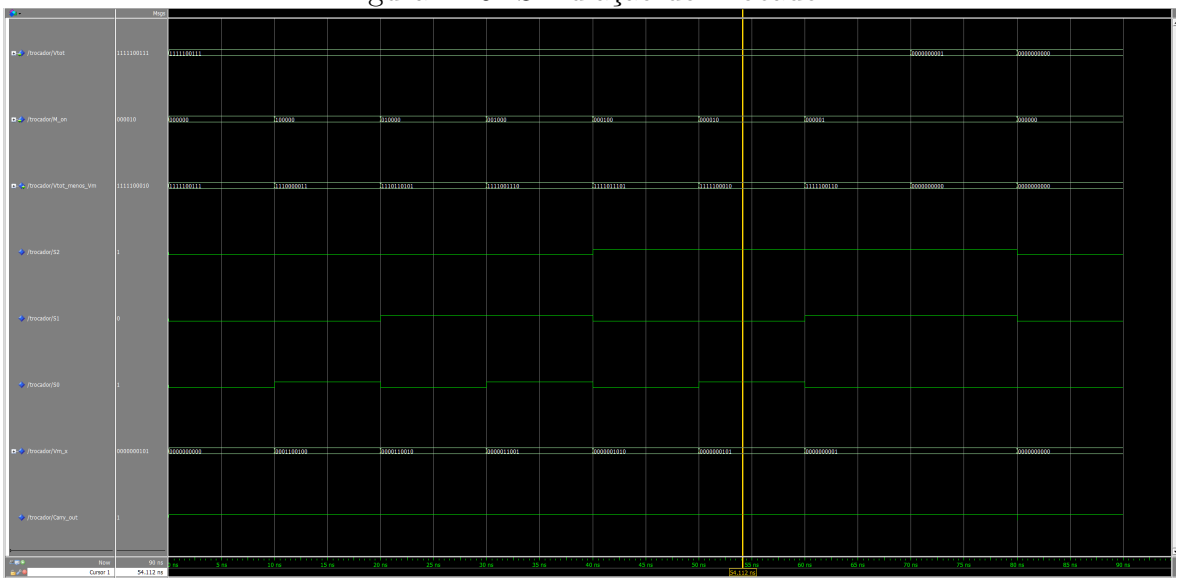
Para compatibilizar a ordem dos seletores, as conexões S0 e S2 foram cruzadas entre o codificador e o multiplexador, mantendo S1. Essa ligação respeita a organização interna do multiplexador utilizado. Quando M_on está em zero, Vm_x também é zero e a saída permanece igual a Vtot. O bloco pressupõe que no máximo uma moeda esteja selecionada por vez, condição garantida pelo Gerador de Operadores.

A Figura 4.15 apresenta a simulação, com os barramentos exibidos em binário. Inicialmente, Vtot recebe 999 e nenhuma moeda está selecionada, resultando em uma saída igual a 999. Entre 10 e 60 ns, os seis bits de M_on são ativados individualmente, a cada 10 ns. São selecionadas, sucessivamente, as moedas de 100, 50, 25, 10, 5 e 1 centavo, produzindo os resultados 899, 949, 974, 989, 994 e 998. Nesse intervalo, Vtot permanece fixo, de modo que cada subtração constitui um teste independente.

Em 70 ns, Vtot é alterado para um centavo, mantendo selecionada a moeda de um centavo. A saída passa a zero, confirmando a subtração entre valores iguais. Em 80 ns, Vtot e M_on recebem zero, mantendo o resultado nulo. O sinal interno Carry_out permanece em nível alto nos casos apresentados, indicando ausência de empréstimo.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Figura 4.15: Simulação do Trocador.



Fonte: Daniel Medeiros.

4.12 Dataway

O Dataway constitui o bloco operacional da máquina de troco, reunindo os componentes responsáveis pelo armazenamento do saldo, pela seleção das moedas e pela atualização dos estoques. Sua implementação é estrutural, conectando os blocos desenvolvidos anteriormente por meio de sinais internos. O módulo recebe os comandos da máquina de estados e retorna as condições Vtot_eq_0, Change_on e Empty_bank, utilizadas nas decisões do controle.

O armazenamento utiliza um registrador de 10 bits para o saldo e seis pares de registradores de quatro bits para os estoques. Cada par contém um banco de referência, que representa a quantidade confirmada, e um banco provisório, atualizado durante a tentativa de composição. Load_Prov copia os valores de referência para os provisórios, enquanto Load_Ref confirma o estoque resultante somente no sucesso. Essa organização permite restaurar os valores provisórios sem alterar o estoque confirmado após uma falha.

A seleção combina as informações do Comparador Vtot e do Contador de Moedas. O primeiro identifica quais denominações não ultrapassam o saldo, enquanto o segundo verifica a existência de estoque e a habilitação dos cofres pelas entradas C1 a C6. Essas condições alimentam o Gerador de Operadores, que seleciona a maior moeda utilizável. O Trocador calcula o novo saldo, e o Verificador de Operação autoriza o decremento do banco correspondente. Para conectar os estoques de quatro bits aos detectores de dez bits, foram acrescentados zeros nas posições superiores.

Os comandos Clear e Change são invertidos para adequar suas polaridades aos registradores. Change controla a sinalização das moedas, enquanto sua negação comanda a atualização do saldo e dos bancos provisórios. Os seis registradores de LED produzem I1 a I6 e recebem uma indicação de erro quando existe saldo restante sem moeda adequada, bloqueada durante as cargas e a reinicialização.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

4.13 Máquina de Estados

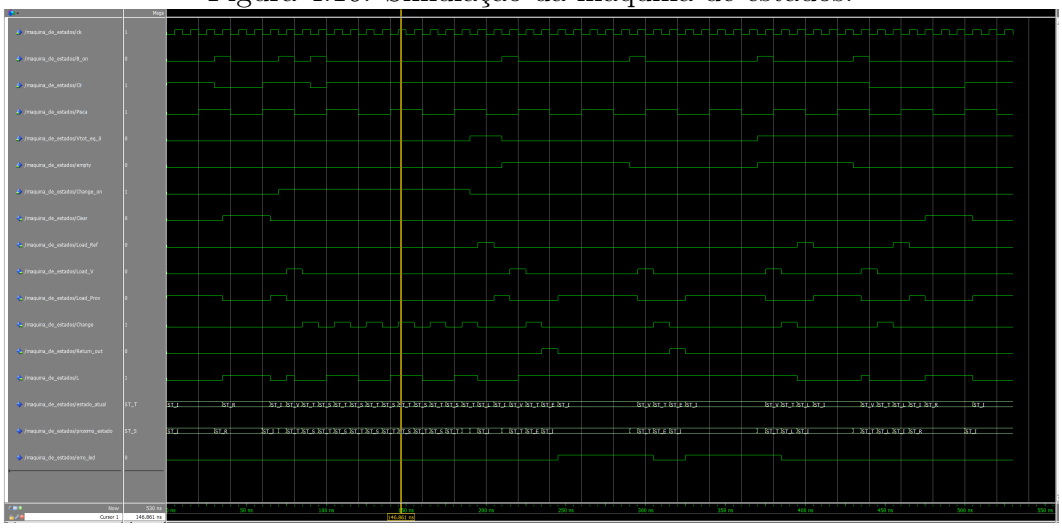
A máquina de estados constitui a unidade de controle da máquina de troco, coordenando as operações do Dataway a partir da solicitação B_on e das condições Vtot_eq_0, empty e Change_on. Suas saídas controlam os registradores, a reinicialização e a operação de troca, além de fornecerem as indicações L e Return_out.

A implementação utiliza sete estados, um processo sequencial para atualizar o estado e a memória de erro, e um processo combinacional para determinar as transições e as saídas. O estado ST_I restaura os bancos provisórios; ST_R comanda a reinicialização; ST_V carrega o valor solicitado; e ST_T e ST_S coordenam a seleção e a atualização das moedas. O sucesso é confirmado em ST_L, enquanto ST_E sinaliza a devolução. Durante o processamento, L acompanha Pisca; após uma falha, permanece aceso até a reinicialização ou o carregamento de nova solicitação. O comando Clr é aceito no estado inicial, sem interromper uma operação em andamento.

A Figura 4.16 apresenta a simulação com período igual a 10 ns. Como exemplo de operação bem-sucedida, em 75 ns a máquina entra em ST_V e ativa Load_V. Em seguida, alterna entre ST_T e ST_S até receber a indicação de saldo zero. Em 195 ns, alcança ST_L e ativa Load_Ref, retornando ao estado inicial em 205 ns.

Como exemplo de falha, em 315 ns a máquina entra em ST_E devido à existência de saldo restante sem moeda utilizável, mesmo com empty em zero. Return_out é acionada e L permanece aceso após o retorno a ST_I. Esse resultado confirma a correção da transição para erro, que deixou de exigir banco vazio, e o funcionamento da memória de sinalização.

Figura 4.16: Simulação da máquina de estados.



Fonte: Daniel Medeiros.

4.14 Máquina de Troco FINAL

O módulo maquina_de_troco_FINAL integra a unidade de controle, o Dataway, o divisor de clock e o circuito de sincronização do botão. Sua função é receber o valor V, selecionar as moedas disponíveis e coordenar a atualização dos estoques, indicando o sucesso ou a impossibilidade de concluir a solicitação.

A implementação conecta os comandos da máquina de estados aos registradores do Dataway e encaminha as condições de operação ao controle. O botão T é sincronizado

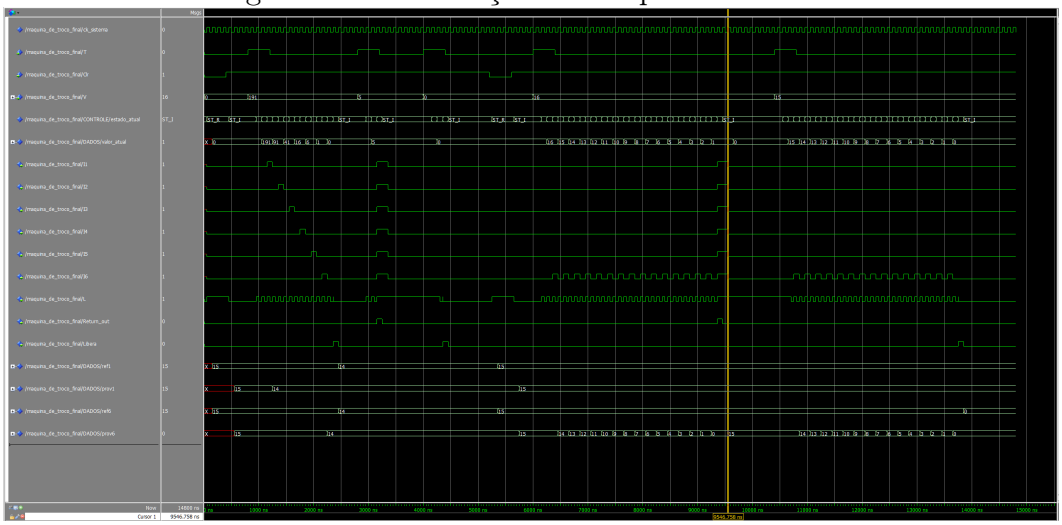
Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

e convertido em um pulso de solicitação, enquanto o clock dividido também fornece a sinalização de pisca. A saída Libera recebe o controle Load_Ref, indicando que o valor foi integralmente composto e que o estoque de referência pode ser atualizado. Em caso de falha, Return_out é acionada e L permanece aceso.

A Figura 4.17 apresenta dois exemplos relevantes. Na solicitação de 191 centavos, são selecionadas moedas de 100, 50, 25, 10, 5 e 1 centavo. O saldo decresce até zero, acompanhado pelos pulsos de I1 a I6. Ao final, Libera é ativada e os bancos de referência observados são atualizados de quinze para quatorze moedas, confirmando o sucesso.

No teste de dezesseis centavos com apenas quinze moedas de um centavo disponíveis, o banco provisório chega a zero enquanto ainda resta um centavo. A máquina sinaliza erro sem ativar Libera. O banco de referência permanece em quinze, e o provisório é restaurado ao retornar ao estado inicial. Esse resultado demonstra que uma tentativa malsucedida não altera o estoque confirmado e o valor não seria repassado ao "cliente"pois estaria faltando 1 centavo.

Figura 4.17: Simulação da Máquina de Troco.



Fonte: Daniel Medeiros.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Capítulo 5

Conclusão

A implementação em VHDL permitiu transformar o projeto RTL da máquina de troco em uma estrutura de hardware organizada em blocos funcionais. As simulações individuais verificaram as operações dos circuitos combinacionais e dos registradores, incluindo carga, retenção, decremento e inicialização, respeitando as prioridades dos controles e as bordas de subida do clock.

Durante a implementação, foram corrigidas inconsistências na comparação dos valores, no controle dos registradores, na seleção das moedas e nas condições de transição da máquina de estados. A simulação integrada demonstrou a composição correta do troco nos casos testados e a preservação do estoque de referência nas tentativas malsucedidas. Também foram verificadas a confirmação do sucesso, a sinalização de devolução e a indicação de processamento e erro pelo LED geral.

Os resultados obtidos confirmam o funcionamento da solução nas condições simuladas e fornecem uma base para sua implementação no kit DE2.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Referências Bibliográficas

DIAS, S. M. Repositório da disciplina ELE3515 – Sistemas Digitais. 2026. Disponível em: https://github.com/sama-md/ele3515/. Acesso em: 28 set. 2026.

TOCCI, R.; WIDMER, N.; MOSS, G. *Digital Systems: Principles and Applications*. [S.l.]: Pearson Education Limited, 2011. ISBN 9780130387936.

VAHID, F. *Sistemas Digitais*. [S.l.]: Grupo A - Bookman, 2009. ISBN 9788577802371.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

Apêndice B

Relato semanal

B.1 Equipe

Tabela B.1: Identificação da equipe.

Função no grupo:	**Nome completo do aluno**	**Responsável por**
Líder:	Mateus Macedo Nogueira	Códigos e Relatório
Redator:	Ananda Dantas Fortunato	Códigos e Relatório
Debatedor:	Daniel Medeiros Soares Carneiro	Códigos e Relatório

Fonte: Produzido pelos autores.

B.2 Defina o problema

A implementação teve como objetivo converter o projeto da máquina de troco do Grupo 3 em descrições VHDL, verificar seu funcionamento por simulação e prepará-lo para a FPGA DE2. Buscou-se preservar a estrutura proposta, corrigindo as inconsistências que impediam a composição correta do troco e a atualização adequada do estoque.

B.3 Registro do brainstorming

No brainstorm, foram discutidas a implementação e a verificação individual dos blocos antes da integração. Foram analisados os controles dos registradores, a seleção das moedas, as transições da máquina de estados e a sinalização de processamento e erro.

B.4 Pontos-chaves

Os pontos-chave foram a correspondência entre os códigos e o projeto, a prioridade dos sinais de controle e a preservação do banco de referência nas tentativas malsucedidas. Também se destacou a necessidade de confirmar o troco somente após a composição integral do valor solicitado.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.

B.5 Questões de Pesquisa

As questões de pesquisa envolveram a tradução dos diagramas para VHDL, a identificação das correções necessárias e a definição de testes capazes de verificar cada operação. Investigou-se também como reconhecer a impossibilidade de continuar o cálculo e como conectar o circuito aos recursos da DE2.

B.6 Planejamento de pesquisa

O planejamento consistiu em consultar o relatório do grupo, o enunciado do professor e o manual da placa, implementar e simular os componentes separadamente e, depois, integrar o circuito completo. Os resultados seriam documentados pela explicação dos blocos e pela análise dos plots, incluindo as correções realizadas e seus efeitos.

Copyright © 2025, ELE3515, Universidade Federal do Rio Grande do Norte.